

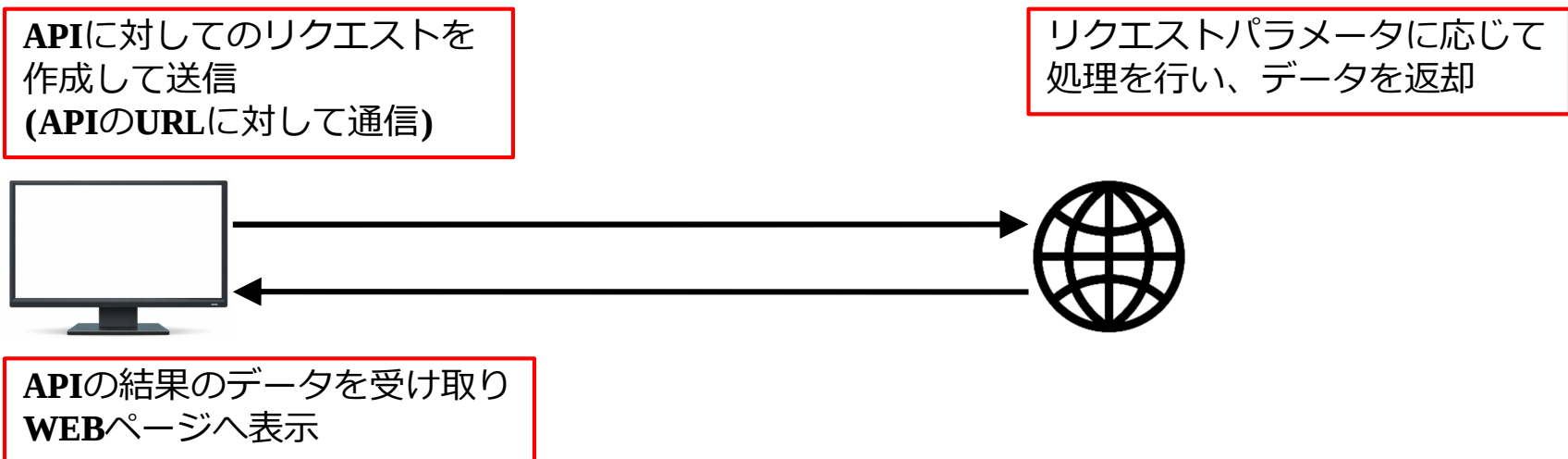
【第6回】 **API**通信と非同期処理について

1. API通信について

1-1. API通信とは？

API通信とは、別のサーバーとデータの送受信を行うことをいい、APIとは**Application Programing Interface**の略である。

APIは異なるプログラム同士がデータのやり取りを行うための受け口となる仕組みのことをいう。



1-2. API通信でできること

JavaScriptでは、一般的なデータベースに直接接続し、データを取得することはできないため、APIを活用し必要なデータを取得する必要がある。

- ・ 一般公開されているデータの取得
- ・ 特定のデータベースからのデータの取得
- ・ 結果に応じたリロードを行わないWEBページの書き換え → Ajax通信

1. API通信について

1-3. API通信の使い方

API通信を行い、データを取得する方法は以下の通りである。

fetch([APIのURL]);

例)

let data = fetch('https://example.com/api'); → API通信により返ってきたデータを変数に格納する

一般的にAPIを利用する際には、**該当のAPIサーバーへアクセスするための認証やトークンキーなどが必要**となるが、認証なしで一部無料で一般公開されているAPIもある。

【一般公開されている無料のAPIの例】

- **OpenStreetMap Nominatim**

→ 地名から緯度・経度を取得できるAPI

<https://nominatim.openstreetmap.org/search?q=Kagoshima&format=json>

- **Open-Meteo**

→ 緯度・経度から天気予報を取得できるAPI

https://api.open-meteo.com/v1/forecast?latitude=35.6895&longitude=139.6917&hourly=weather_code

- **Cats API**

→ 猫の画像を取得できるAPI

<https://api.thecatapi.com/v1/images/search>

ただし、無料のAPIの場合、サービスの変更に伴う公開終了や有料化というような変更もあるため、利用する際には注意が必要である。

1. API通信について

【例題1】

ブラウザのアドレスバーに以下のURLを入力し、APIを実行しなさい。

① かわいい猫

<https://api.thecatapi.com/v1/images/search>

→ 結果が返ってきたら、戻り値のJSON内の"url"の値をコピーして、ブラウザの別タブでURLを表示する。

② ポケモンAPI

<https://pokeapi.co/api/v2/pokemon-species/1/>

→ 対象のポケモンの日本語名を探す。

2. 非同処理について

2-1. 非同期処理とは?

非同期処理とは、実行中の処理完了を待たずに次の処理へ進む処理方法のことで、API通信で利用するfetchは原則非同期処理となる。

APIでの処理結果(データの戻り)を待って、結果に応じて処理を行うためには**fetch**の構造を理解し、同期的に処理を行うように実装する必要がある。

2-2. 非同期処理を扱う方法

非同期処理を同期的に処理するように実装する方法としては、以下の2種類の方法がある。

【Promise形式】→ .then()を使用する

```
fetch([APIのURL])  
  .then([APIの結果が正常に返ってきた場合の処理])  
  .catch([APIでエラーが発生した場合の処理]);
```

上記のように、**.then**を利用すると前に書いた処理が完了した後に、処理の結果を受け取り指定した処理を行うことが可能となる。

.thenや**.catch**の後に記述する処理のことを、コールバック関数といい、.thenや.catchの後にはfunctionを定義し、関数化して処理を実行する。

定義したコールバック関数のパラメータとして、APIで返ってきたデータを受け取り、**function**の中で使用することができる。

2. 非同処理について

2-2. 非同期処理を扱う方法

例)

```
fetch('https://www.example.com/api') → APIを実行
  .then(function(response) → APIの処理が成功した際の処理を定義
  {
    return response.json(); → APIの戻り値をJSON化する
  })
  .then(function(data) → 1つ目のコールバック関数が完了した後の処理を定義
  {
    alert(data); → APIの戻り値をJSON文字列化したものをダイアログ表示
  })
  .catch(function(error) → APIの処理でエラーが発生した際の処理を定義
  {
    alert(error); → APIの処理でのエラーの内容をダイアログ表示
  });
```

Promise形式で記述すると、成功時の処理と失敗時の処理が明示的でわかりやすく、成功時の処理を並列化しやすいという特徴がある。

その反面、成功時の並列化が複雑になりやすかったり、各処理がネストする形となるため、可読性が落ちてしまう。

2. 非同期処理について

2-2. 非同期処理を扱う方法

【**await / async**形式】

Promise形式と同様に、処理の結果を待つて次の処理を行う記述として、**await・asyncを利用するとプログラムの可読性が上がり、記述した順にプログラムが実行される**ためわかりやすい。

async function [関数名]() → **async function**で定義すると、呼び出し側は処理完了を待つて次の処理を行う
{
 [関数内で行う処理]
}

await [関数名](); → 呼び出し側は**await**を付けて呼び出すと、呼び出した関数の処理が終わるまで待機する

例)

```
let api_result_data = await getData();

alert(api_result_data);

async function getData()
{
  let result = await fetch('https://www.example.com/api');

  let data = await result.json();

  return data;
}
```

2. 非同期処理について

【例題2】

Javascript-sample-q2.htmlのボタンをクリックした際に、

<https://api.thecatapi.com/v1/images/search>

のAPIを実行して結果のURLの画像をimgタグに表示するJavaScriptを実装しなさい。

【例題3】

Javascript-sample-q3.htmlのボタンをクリックした際に、

<https://nominatim.openstreetmap.org/search?q=Kagoshima&format=json>

のAPIを実行して、取得した緯度経度とOpen APIのOpen-Meteoを利用して、現在の天気を表示するJavaScriptを実装しなさい。

※ Open-Meteoの使用方法 (英語版のみ)

<https://open-meteo.com/en/docs>

【Open-Meteo 天気コード一覧】

コード	天気	コード	天気	コード	天気	コード	天気
0	快晴	55	霧雨（強）	71	雪（弱）	86	にわか雪（強）
1	ほぼ晴れ	56	凍る霧雨（弱）	73	雪（中）	95	雷雨
2	部分的にくもり	57	凍る霧雨（強）	75	雪（強）	96	ひょうを伴う雷雨（弱）
3	くもり	61	雨（弱）	77	霧雪（細かい雪）	99	ひょうを伴う雷雨（強）
45	霧	63	雨（中）	80	にわか雨（弱）		
48	着氷性の霧	65	雨（強）	81	にわか雨（中）		
51	霧雨（弱）	66	凍る雨（弱）	82	にわか雨（強）		
53	霧雨（中）	67	凍る雨（強）	85	にわか雪（弱）		